

AI-TECH

Przykład 1 DO LABORATORIUM 1
z UCZENIA MASZYNOWEGO

Regresja liniowa i logistyczna

Tomasz Neumann

Zagadnienie regresji liniowej - przykłady

Rzeczpospolita
PolskaUnia Europejska
Europejski Fundusz
Rozwoju Regionalnego

Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020,
Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2
"Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej" Tytuł projektu: „Akademia
Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

1. Przykład 1

Regresja liniowa

Jest to podejście, które zakłada liniową zależność pomiędzy zmiennymi wejściowymi, a zmienną docelową. Najprostszym typem tego rodzaju regresji jest przyjęcie modelu z jedną zmienną wejściową x i zmienną wyjściową y . Celem modelu jest znalezienie parametrów funkcji, która na podstawie wartości x_i będzie zwracała wartości y_i

$$y = a_0 + a_1x,$$

gdzie a_0 to wyraz wolny (intercept), a współczynnik a_1 (slope), określa o ile zmienia się wartość y_i wraz ze zmianą x_i . Uczenie takiego modelu polega na znalezieniu wystarczająco dobrego przybliżenia funkcji zmiennej wyjściowej do zmiennych wejściowych, odpowiadającej hipotezie bliskiej prawdziwej. W celu zamodelowania takiej sytuacji wygenerujemy dane, które mogłyby opisywać zmianę położenia ciała w ruchu jednostajnie prostoliniowym. Na początku należy zaimportować biblioteki, które będą potrzebne do realizacji zadania.

```
In [1]: # Import podstawowych bibliotek języka Python
import numpy as np # operacje numeryczne
import matplotlib # wykresy i ich ustawienia
import matplotlib.pyplot as plt # tworzenie wykresu
import time #diagnostyka czasu wykonywania operacji
```

W kolejnym kroku zakłada się współczynniki idealnego modelu oraz generuje się szum gaussowski imitujący zakłócenia o pewnej amplitudzie występujące podczas wykonywania pomiaru.

```
In [2]: # Założymy, że zjawisko które chcemy zamodelować opisywane jest następującą zależ
# np. modelowanie zmiany położenia ciała w ruchu jednostajnie prostoliniowym
# y = 0.2* x + 0.4 + noise
# noise jest czynnikiem zakłócającym pomiar

Wt = 0.2
bt = 0.4
```

```
# deklaracja liczby zmiennych używanych podczas generacji danych
n_samples = 100

# ustawienie wartości zaszumienia na wartość 0
noise = 0
# jeżeli jednak ma być ustawiona pewna wartość szumu z dowolną amplitudą, należy
noise = np.random.rand(n_samples).astype(np.float32)/10
```

W tej części inicjowana jest zmienna niezależna oraz zmienna zależna oraz wyświetlone zostaną poglądowo wygenerowane wartości.

```
In [3]: # Generacja losowej zmiennej niezależnej x
x_t = np.random.rand(n_samples).astype(np.float32)

# posortowanie wygenerowanych wartości x w celu lepszej prezentacji danych na wy
x_training = np.sort(x_t)

# Wyliczenie wartości dla zmiennej zależnej y
y_training = Wt * x_training + bt + noise

# Diagnostyczne wyświetlenie wygenerowanych wartości
print("\n x_training =", x_training)
print("\n y_training =", y_training)
print("\n Noise std. dev. =", np.std(noise))
```

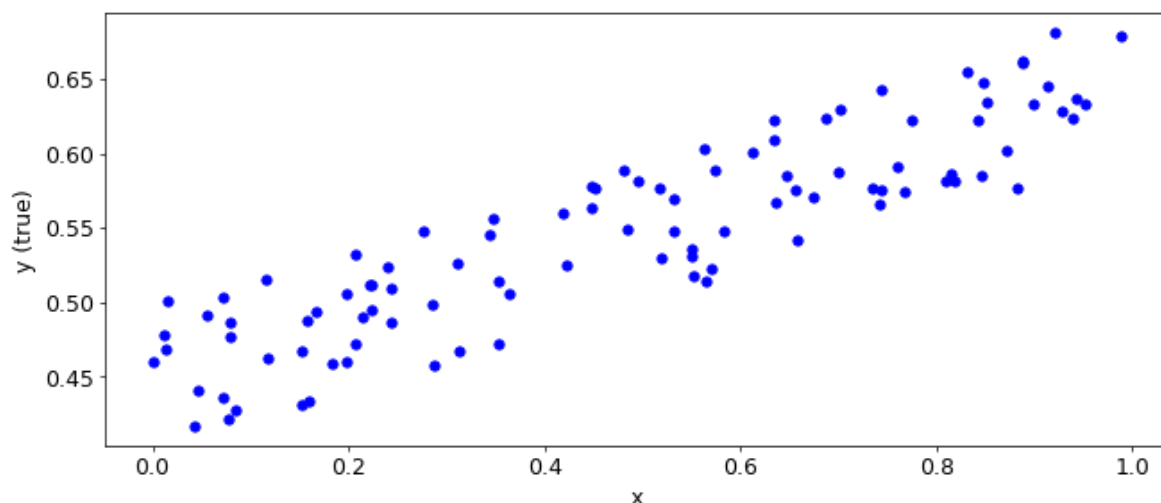
```
x_training = [0.0013215  0.01258633 0.01277184 0.01636658 0.0425447  0.0465515
0.05587009 0.07181977 0.07242525 0.0771939  0.07954455 0.07995455
0.08568605 0.11679037 0.11748587 0.15261117 0.15272623 0.15771624
0.15914747 0.16740549 0.18343747 0.19793798 0.19889264 0.20658731
0.20771796 0.21446536 0.22159152 0.22314264 0.2242192  0.24033502
0.24309398 0.24336027 0.27690345 0.28636116 0.28781566 0.31107605
0.3128281  0.3433668  0.3482004  0.35245734 0.35410908 0.36419684
0.4186239  0.42279863 0.44761768 0.4487231  0.4524158  0.48113245
0.48424074 0.49646768 0.51747376 0.5185985  0.53146225 0.5326375
0.5499292  0.5507654  0.5517178  0.56395006 0.56534183 0.57107127
0.57405704 0.58408105 0.6121756  0.6337999  0.63513285 0.6358114
0.6467832  0.65683204 0.6573646  0.67437196 0.687647  0.6996211
0.7010135  0.7355195  0.7423845  0.74373585 0.74475455 0.76099515
0.76773196 0.7751783  0.81014496 0.81484216 0.81837136 0.8323711
0.8429348  0.84565765 0.84740645 0.851612  0.8708825  0.88323677
0.887754  0.8883415  0.89827776 0.91331476 0.92156494 0.9278028
0.93929845 0.9437155  0.951373  0.9891319 ]
```

```
y_training = [0.45976594 0.47844112 0.46807772 0.50084305 0.41685244 0.4407944
0.49091804 0.43561023 0.50388324 0.42183587 0.48705864 0.47703072
0.42799392 0.5156088  0.46219677 0.43163136 0.46727017 0.48723096
0.43397507 0.49431965 0.458953  0.5055114  0.45945638 0.53201926
0.4718605  0.49025628 0.51194906 0.51186377 0.49512026 0.5238824
0.5091736  0.48680326 0.5477254  0.49872267 0.4581966  0.5268867
0.4668527  0.54519564 0.55586547 0.51389855 0.47168896 0.50631744
0.55997413 0.52455485 0.5781599  0.56328094 0.57698536 0.58922285
0.5497016  0.58156186 0.57681376 0.53036875 0.5700165  0.547452
0.5312887  0.5356441  0.5182421  0.60323  0.5148186  0.52318436
0.5893214  0.54785347 0.6013064  0.6228911  0.60992897 0.5667879
0.58500355 0.57583976 0.5421873  0.57110476 0.62331146 0.5872235
0.63031024 0.5769029  0.56649494 0.64302146 0.57609886 0.5915164
0.5746595  0.6225153  0.58204  0.5863106 0.581494  0.6551406
0.62278044 0.58582497 0.6479531  0.6348889 0.6025453  0.5772177
0.6626514  0.66101426 0.6340812  0.6456254 0.68144685 0.6289416
0.62334543 0.63771236 0.6334533  0.67947334]
```

Noise std. dev. = 0.0289496

W tym kroku został zaprezentowany przykładowy kod umożliwiający wizualizację wygenerowanych danych w poprzednim punkcie.

```
In [4]: # Wizualizacja wygenerowanych danych
# Ustawienia związane z wyświetlaniem
matplotlib.rcParams.update({'font.size': 14})
f = plt.figure(figsize=(12, 5), dpi=64)
f1 = f.add_subplot(111)
f1.set_xlabel('x')
f1.set_ylabel('y (true)')
# Wykreślenie danych
f1.plot(x_training, y_training, 'bo')
plt.show()
```



W postaci wektorowej, predykcja za pomocą modelu regresji liniowej może zostać zapisana jako

$$y = h_{\theta}(\mathbf{x}) = \theta \cdot \mathbf{x}$$

gdzie θ to wektor parametrów modelu (wraz z punktem obciążenia θ_0 i $\theta_1, \dots, \theta_n$), $\mathbf{x}$ to wektor cech danego przykładu, zaś h_{θ} to funkcja hipotezy, wykorzystująca parametry θ modelu. Aby wyznaczyć parametry, które będą optymalnie dopasowane do danych zbioru uczącego należy zastosować miarę wydajności dla modelu regresyjnego, a tu najlepiej sprawdza się pierwiastek błędu średniokwadratowego RMSE. W praktyce łatwiej jest jednak minimalizować wartość błędu średniokwadratowego MSE, który można wyliczać jako

$$MSE(\mathbf{X}, h_{\theta}) = \frac{1}{m} \sum_{i=1}^m (|\theta^T \mathbf{x}^{(i)} - y^{(i)}|)^2$$

Do określenia wartości wektora θ minimalizującego funkcję kosztu używa się jawnego rozwiązania w postaci równania normalnego

$$\hat{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \cdot \mathbf{X}^T \cdot \mathbf{y},$$

gdzie $\hat{\theta}$ oznacza wartość wektora θ minimalizująca funkcję kosztu, zaś $\mathbf{y}$ jest wektorem wartości docelowych. Poniższy fragment kodu przedstawia obliczenie wartości $\hat{\theta}$ za pomocą równania normalnego.

```
In [7]: # Dodanie do każdej próbki x0=1
X_b = np.c_[np.ones((100, 1)), x_training]
start = time.time()
hat_theta = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y_training)
dt = time.time() - start;
print('Upłynęło ', dt*1000, ' ms')
print(hat_theta)
```

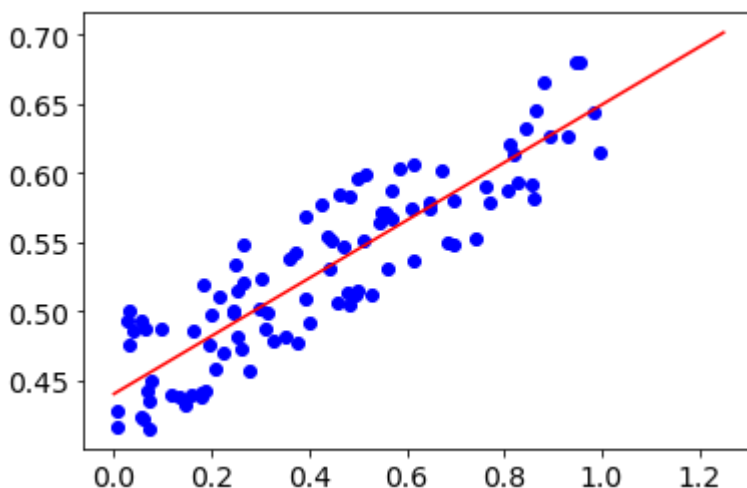
```
Upłynęło 0.9996891021728516 ms
[0.45012637 0.2017739 ]
```

gdzie pierwsza wartość z tablicy informuje nas o wartości wyrazu wolnego, a druga wartość niesie informację o współczynniku kierunkowym wyznaczonej prostej. Na

podstawie wyznaczonych wartości parametrów prostej można dokonać predykcji jak i wizualizację uzyskanych wyników na tle danych wejściowych.

```
In [6]: # Wyliczenie prognozy na podstawie hat_theta
x_predict = np.array([[0], [1.25]])
x_predict_b = np.c_[np.ones((2,1)), x_predict]
y_predict = x_predict_b.dot(hat_theta)
print(y_predict)
# zilustrowanie rozwiązań na wykresie
plt.plot(x_training, y_training, "bo") #dane treningowe
plt.plot(x_predict, y_predict, "r") # dane z predykcji
plt.show()
```

[0.44060449 0.70142127]



Wykonanie tych samych operacji można wykonać za pomocą biblioteki Scikit-Learn wykorzystując klasę LinearRegression, która jest opisana w dokumentacji pod adresem https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

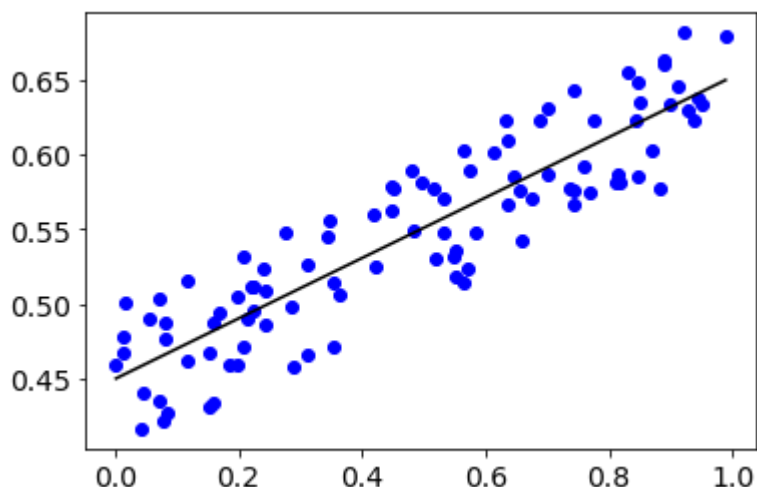
Wywołanie modelu może odbywać się poprzez przededefiniowanie parametrów domyślnych takich jak:

- `fit_intercept` (domyślnie `True`), oblicza punkt przecięcia z osią. W przypadku ustawienia na `False`, oczekuje się, że dane będą wyśrodkowane.
- `normalize` (domyślnie `False`) dokonuje normalizacji danych wejściowych, gdy `fit_intercept` jest ustawiony na `True`, w przeciwnym razie jest ignorowany. Warto sprawdzić podczas testów, czy włączenie normalizacji poprawi ostatecznie wyniki końcowe.

```
In [8]: from sklearn.linear_model import LinearRegression
#należy przygotować odpowiednio dane zgodnie z opisem
#https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRe
lin_reg = LinearRegression() #można sprawdzić, czy wynik poprawi się przy zastos
X_c = x_training.reshape(100,1)
noise_c = noise.reshape(100,1)
y_c = 0.2*X_c+0.4+noise_c
start = time.time()
lin_reg.fit(X_c, y_c)
dt = time.time() - start;
print('Upłynęło ',dt*1000,' ms')
```

```
print(lin_reg.intercept_, lin_reg.coef_)
plt.plot(X_c, y_c, "bo") # dane treningowe
plt.plot(X_c, lin_reg.intercept_+lin_reg.coef_*X_c, "k") #wykres liniowy na pods
plt.show()
```

Upłynęło 0.9996891021728516 ms
 [0.4501263] [[0.20177393]]



Wyświetlenie współczynników w tym przypadku odbyło się poprzez wywołanie atrybutu *coef_*, natomiast wyrazu wolnego za pomocą atrybutu *intercept_*. Do wyliczenia prognozy można posłużyć się metodą *predict* do której należy przekazać tablicę danych próbek.

```
In [9]: print(lin_reg.predict(np.array([[0.5],[1],[100]])))
```

```
[[ 0.55101325]
 [ 0.65190022]
 [20.62751895]]
```

Dla modelu można wywołać metodę *score*, która zwraca wartość współczynnika determinacji *R*. Najlepszy wynik ma wartość 1.0 oraz może być ujemny (model może być arbitralnie gorszy). Model, który zawsze będzie przewidywał wartość oczekiwaną zmiennej zależnej, pomijając cechy wejściowe miałby wartość 0 dla współczynnika determinacji.

```
In [10]: print('R = ', lin_reg.score(X_c, y_c))
```

```
R = 0.805990445956372
```

W kolejnym kroku wykorzystamy możliwości modułu *statsmodels*, który za pomocą metody najmniejszych kwadratów OLS umożliwia oszacowanie wartości parametrów modelu regresji (<https://www.statsmodels.org/stable/regression.html>). Oprócz zaimportowania odpowiedniego pakietu należy tu także prawidłowo przygotować i wywołać dane w metodzie uczącej. Użytkownik musi tu zadbać o uwzględnienie punktu przecięcia (wyrazu wolnego, obciążenia) poprzez zastosowanie metody *add_constant*. Także przy wywołaniu metody OLS należy pamiętać, że pierwszym parametrem jest tablica ze zmienną niezależną, jako druga jest podawana tablica $n \times k$ z liczbą obserwacji n i regresorów k .

```
In [11]: import statsmodels.api as sm
X_d = x_training.reshape(100,1)

noise_d = noise.reshape(100,1)
y_d = 0.2*X_d+0.4+noise_d

X_d = sm.add_constant(X_d, prepend=False)
mod = sm.OLS(y_d, X_d)
res = mod.fit()

pred = res.predict(np.array([[0.5, 1]]))
print(pred)
```

[0.55101332]

Moduł ten posiada zaawansowane możliwości prezentacji różnego rodzaju statystyk za pomocą metod *summary* oraz *summary2* dla wyznaczonych współczynników.

```
In [21]: print(res.params)
print(res.summary2())
```

[0.2017739 0.45012637]

Results: Ordinary least squares

```
=====
Model:                OLS                Adj. R-squared:    0.804
Dependent Variable: y                AIC:                -420.6842
Date:                2022-03-26 15:02    BIC:                -415.4739
No. Observations:    100                Log-Likelihood:    212.34
Df Model:            1                  F-statistic:       407.1
Df Residuals:        98                  Prob (F-statistic): 1.13e-36
R-squared:           0.806                Scale:           0.00085491
=====
```

	Coef.	Std.Err.	t	P> t	[0.025	0.975]
x1	0.2018	0.0100	20.1775	0.0000	0.1819	0.2216
const	0.4501	0.0056	80.1213	0.0000	0.4390	0.4613

```
-----
Omnibus:                34.936                Durbin-Watson:           2.241
Prob(Omnibus):          0.000                Jarque-Bera (JB):        6.264
Skew:                   -0.108                Prob(JB):                0.044
Kurtosis:               1.793                Condition No.:          4
=====
```

Zadania do wykonania

Zadanie 1

Przeprowadź eksperymenty dla różnych wartości zaszumienia i szerokości przedziałów zmiennej X - porównaj czasy wykonywania operacji wyliczania współczynników oraz opisz ten wpływ na wartości uzyskiwanych parametrów statystyk.

